SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO1: *Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05 **Total Marks:100**

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

I M.Gayathri(192472074) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Gayathri

Annexure A

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No

P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes
P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

3. Retail Sales Dataset
Question 3
A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

4. Banking Loan Approval Dataset
Question 4
A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved

C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected
C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

5. Agriculture Crop Yield Dataset

Question 5

An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800
F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

Common EDA Tasks:

1. Identify the number of rows and columns.
2. Check data types of each column.
3. Find missing values and duplicate values.
4. Calculate mean, median, minimum, maximum, and standard deviation.
5. Draw suitable charts such as bar chart, histogram, scatter plot, and box plot.
6. Identify important patterns from the dataset.

7. Write 4 to 5 observations based on EDA.
8. Give a short conclusion from the analysis.

RUBRICS FOR CALCULATION

Exploratory Data Analysis-Based Practical Assessment					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Dataset Understanding	20	Clearly explains dataset source, features, target variables, data types, and problem context with strong clarity	Explains dataset, features, and variables with adequate clarity	Basic explanation of dataset with limited understanding of variables	Poor understanding of dataset; variables and problem context are unclear
Data Cleaning	25	Effectively identifies and handles missing values, duplicates, incorrect entries, and outliers using suitable methods	Handles most missing values, duplicates, and errors correctly	Performs basic cleaning but misses some important data quality issues	Minimal or incorrect data cleaning; major errors remain in the dataset
Data Analysis and Statistical Summary	20	Provides detailed descriptive statistics, comparisons, patterns, and meaningful interpretations	Provides relevant summary statistics and basic interpretation	Provides limited statistics with weak explanation	Lacks statistical analysis or gives incorrect interpretation
Data Visualization	20	Uses highly appropriate charts with proper labels, titles, and clear visual interpretation	Uses suitable charts with mostly clear labels and explanation	Uses limited or basic charts with unclear interpretation	Poor or incorrect visualizations; charts do not support analysis
Insight Generation and Reporting	15	Presents strong insights, conclusions, and recommendations based on data evidence	Presents clear findings and conclusions with reasonable support	Provides basic findings but lacks depth or proper justification	Findings are unclear, unsupported, or not related to the data

Question 1: Education Analytics Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Education Analytics Dataset
data = {
    'Student_ID': ['S01','S02','S03','S04','S05','S06','S07','S08','S09','S10',
                   'S11','S12','S13','S14','S15','S16','S17','S18','S19','S20'],
    'Attendance': [92,55,78,60,88,47,95,72,50,83,66,90,58,76,62,85,45,80,68,53],
    'Study_Hours': [5,2,4,3,5,1,6,3,2,4,3,5,2,4,3,5,1,4,3,2],
    'Internal_Marks': [45,24,38,28,42,20,48,35,22,40,30,44,25,36,29,41,18,39,32,23],
    'Result': ['Pass','Fail','Pass','Fail','Pass','Fail','Pass','Pass','Fail','Pass',
               'Fail','Pass','Fail','Pass','Fail','Pass','Fail','Pass','Pass','Fail']
}

df = pd.DataFrame(data)

# 1. Number of rows and columns
print("\n1. NUMBER OF ROWS AND COLUMNS")
print(df.shape)

# 2. Data types
print("\n2. DATA TYPES")
print(df.dtypes)

# 3. Missing values and duplicates
print("\n3. MISSING VALUES")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# Data cleaning
df = df.drop_duplicates()
df = df.dropna()

# 4. Statistical summary
print("\n4. STATISTICAL SUMMARY")
print(df[['Attendance', 'Study_Hours', 'Internal_Marks']].agg(
    ['mean', 'median', 'min', 'max', 'std']
).round(2))

# 5. Bar chart
df.groupby('Result')['Internal_Marks'].mean().plot(kind='bar')
plt.title("Average Internal Marks by Result")
plt.xlabel("Result")
plt.ylabel("Average Marks")
plt.tight_layout()
```

```
plt.show()

# Histogram
plt.hist(df['Internal_Marks'], bins=6, edgecolor='black')
plt.title("Distribution of Internal Marks")
plt.xlabel("Internal Marks")
plt.ylabel("Number of Students")
plt.show()

# Scatter plot
sns.scatterplot(data=df, x='Attendance',
                y='Internal_Marks', hue='Result')
plt.title("Attendance vs Internal Marks")
plt.tight_layout()
plt.show()

# Box plot
sns.boxplot(data=df, x='Result', y='Study_Hours')
plt.title("Study Hours by Result")
plt.tight_layout()
plt.show()

# 6. Important patterns
print("\n6. IMPORTANT PATTERNS")
print(df.groupby('Result')[
    ['Attendance', 'Study_Hours', 'Internal_Marks']
].mean().round(2))

print("\nCorrelation:")
print(df[['Attendance', 'Study_Hours',
          'Internal_Marks']].corr().round(2))

# 7. Observations
print("\n7. OBSERVATIONS")
print("1. The dataset contains 20 students.")
print("2. Average attendance is 70.15.")
print("3. Average study hours are 3.35.")
print("4. Average internal marks are 32.95.")
print("5. Students who pass generally have higher attendance, study hours, and internal
marks.")

# 8. Conclusion
print("\n8. CONCLUSION")
print("Attendance, study hours, and internal marks are associated with student results.")
print("Regular attendance and consistent study may help improve academic performance.")
```

OUTPUTS:

Q1.

```
1. Number of rows and columns:
(20, 5)
```

Q2.

```
2. Data types:
Student_ID          object
Attendance           int64
Study_Hours          int64
Internal_Marks       int64
Result              object
dtype: object
```

Q3.

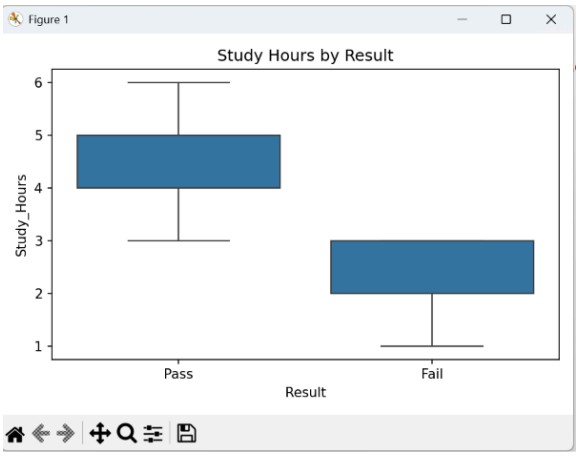
```
3. Missing values:
Student_ID          0
Attendance          0
Study_Hours         0
Internal_Marks      0
Result              0
dtype: int64

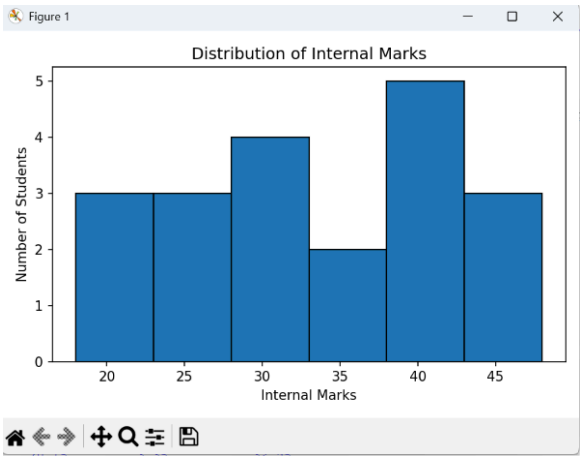
Duplicate rows:
0
```

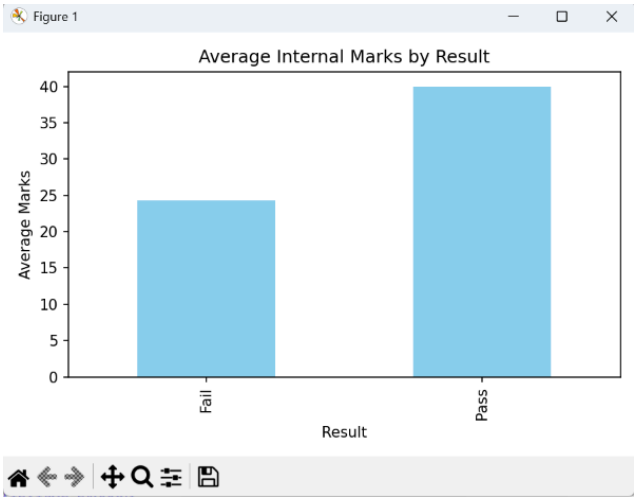
Q4.

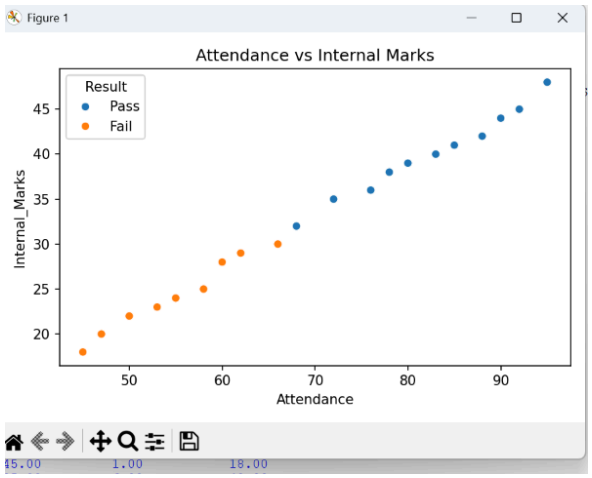
```
4. Statistical summary:
        Attendance  Study_Hours  Internal_Marks
mean     70.150000     3.350000       32.950000
median   70.000000     3.000000       33.500000
min      45.000000     1.000000       18.000000
max      95.000000     6.000000       48.000000
std      15.938286     1.424411        9.087151
```

Q5.









Q6.

```
6. IMPORTANT PATTERNS
        Attendance  Study_Hours  Internal_Marks
Result
Fail         55.11         2.11           24.33
Pass         82.45         4.36           40.00

Correlation:
               Attendance  Study_Hours  Internal_Marks
Attendance           1.00         0.98            1.00
Study_Hours          0.98         1.00            0.98
Internal_Marks       1.00         0.98            1.00
```

Q7.

```
7. OBSERVATIONS
1. The dataset contains 20 students.
2. Average attendance is 70.15.
3. Average study hours are 3.35.
4. Average internal marks are 32.95.
5. Students who pass generally have higher attendance, study hours, and internal marks.
```

Q8.

```
8. CONCLUSION
Attendance, study hours, and internal marks are associated with student results.
Regular attendance and consistent study may help improve academic performance.
```

QUESTION 2:Healthcare Analytics Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Healthcare Analytics Dataset
data = {
    'Patient_ID': ['P01','P02','P03','P04','P05','P06','P07','P08','P09','P10',
                   'P11','P12','P13','P14','P15','P16','P17','P18','P19','P20'],
    'Age': [45,32,55,28,60,40,52,35,48,30,62,38,50,27,58,42,46,34,53,29],
    'Sugar_Level':
[145,98,180,90,200,120,165,105,155,95,210,115,170,88,190,130,150,100,175,92],
    'BP':
[130,118,145,110,150,125,140,120,135,115,155,122,142,108,148,128,132,116,144,112],
```

```
    'BMI': [28,23,31,22,34,26,30,24,29,23,35,25,30,21,33,27,28,24,31,22],
    'Disease_Status': ['Yes','No','Yes','No','Yes','No','Yes','No','Yes','No',
                       'Yes','No','Yes','No','Yes','No','Yes','No','Yes','No']
}
df = pd.DataFrame(data)
# 1. Number of rows and columns
print("\n1. NUMBER OF ROWS AND COLUMNS")
print(df.shape)
# 2. Check data types
print("\n2. DATA TYPES")
print(df.dtypes)
# 3. Missing values and duplicates
print("\n3. MISSING VALUES")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# Data cleaning
df = df.drop_duplicates()
df = df.dropna()
# 4. Statistical summary
print("\n4. STATISTICAL SUMMARY")
print(df[['Age', 'Sugar_Level', 'BP', 'BMI']].agg(
    ['mean', 'median', 'min', 'max', 'std']
).round(2))
# 5(a). Bar chart
df.groupby('Disease_Status')[['Sugar_Level', 'BP', 'BMI']].mean().plot(
    kind='bar', figsize=(8, 5)
)
plt.title("Average Health Measurements by Disease Status")
plt.ylabel("Average Value")
plt.tight_layout()
plt.show()
# 5(b). Histogram
plt.hist(df['Sugar_Level'], bins=6, edgecolor='black')
plt.title("Distribution of Blood Sugar Levels")
plt.xlabel("Sugar Level")
plt.ylabel("Number of Patients")
plt.show()
# 5(c). Scatter plot
sns.scatterplot(data=df, x='Age', y='Sugar_Level',
                hue='Disease_Status')
plt.title("Age vs Blood Sugar Level")
plt.tight_layout()
plt.show()
# 5(d). Box plot
sns.boxplot(data=df, x='Disease_Status', y='BMI')
plt.title("BMI by Disease Status")
plt.show()
# 6. Identify important patterns
print("\n6. IMPORTANT PATTERNS")
```

```
print(df.groupby('Disease_Status')[
    ['Age', 'Sugar_Level', 'BP', 'BMI']
].mean().round(2))
print("\nCorrelation:")
print(df[['Age', 'Sugar_Level', 'BP', 'BMI']].corr().round(2))
# 7. Observations
print("\n7. OBSERVATIONS")
print("1. The dataset contains 20 patient records.")
print("2. Patients marked Yes have higher blood sugar levels in this dataset.")
print("3. Patients marked Yes generally have higher blood pressure.")
print("4. Patients marked Yes generally have higher BMI.")
print("5. The disease-positive group has a higher average age.")
# 8. Conclusion
print("\n8. CONCLUSION")
print("EDA identifies associations between disease status, age,")
print("blood sugar, blood pressure, and BMI.")
print("These observations are exploratory and do not establish a diagnosis.")
```

OUTPUTS:

Q1.

```
1. NUMBER OF ROWS AND COLUMNS
(20, 6)
```

Q2.

```
2. DATA TYPES
Patient_ID        object
Age                int64
Sugar_Level        int64
BP                 int64
BMI                int64
Disease_Status    object
dtype: object
```

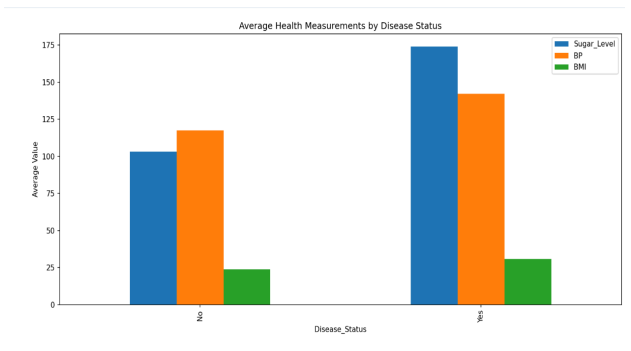
Q3.

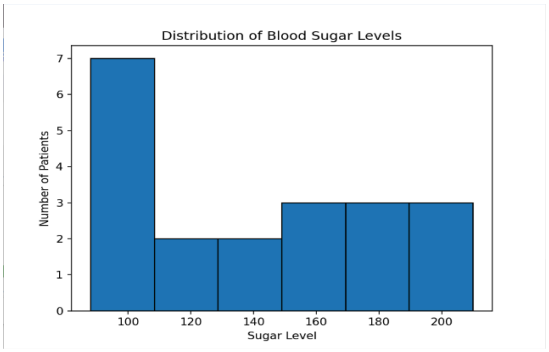
```
3. MISSING VALUES
Patient_ID        0
Age               0
Sugar_Level       0
BP                0
BMI               0
Disease_Status    0
dtype: int64
Duplicate rows: 0
```

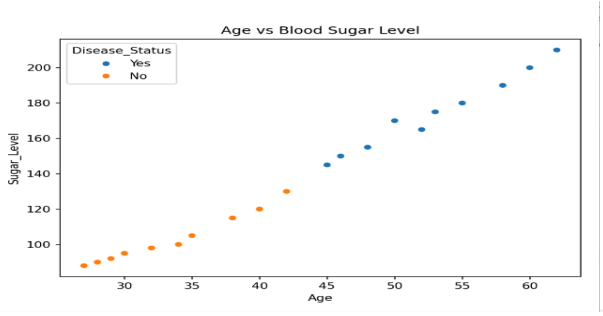
Q4.

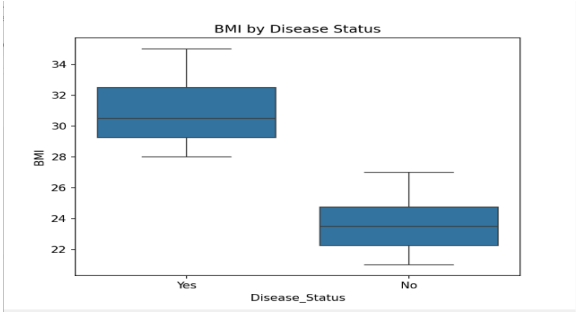
```
4. STATISTICAL SUMMARY
              Age  Sugar_Level      BP     BMI
mean        43.20       138.65  129.75   27.30
median      43.50       137.50  129.00   27.50
min         27.00        88.00  108.00   21.00
max         62.00       210.00  155.00   35.00
std         11.32        40.33   14.52    4.26
```

Q5









Q6.

```
6. IMPORTANT PATTERNS
                 Age  Sugar_Level     BP   BMI
Disease_Status
No              33.5        103.3  117.4  23.7
Yes             52.9        174.0  142.1  30.9

Correlation:
              Age  Sugar_Level     BP   BMI
Age          1.00         0.99   0.99  1.00
Sugar_Level  0.99         1.00   0.99  0.99
BP           0.99         0.99   1.00  0.99
BMI          1.00         0.99   0.99  1.00
```

Q7.

```
7. OBSERVATIONS
1. The dataset contains 20 patient records.
2. Patients marked Yes have higher blood sugar levels in this dataset.
3. Patients marked Yes generally have higher blood pressure.
4. Patients marked Yes generally have higher BMI.
5. The disease-positive group has a higher average age.
```

Q8.

```
8. CONCLUSION
EDA identifies associations between disease status, age,
blood sugar, blood pressure, and BMI.
These observations are exploratory and do not establish a diagnosis.
```

QUESTION 3:Retail Sales Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Retail Sales Dataset
```

```
data = {
   'Product_ID': ['PR01','PR02','PR03','PR04','PR05','PR06','PR07','PR08','PR09','PR10',
                  'PR11','PR12','PR13','PR14','PR15','PR16','PR17','PR18','PR19','PR20'],
   'Category': ['Grocery','Snacks','Dairy','Fruits','Vegetables',
                'Grocery','Snacks','Dairy','Fruits','Vegetables',
                'Grocery','Snacks','Dairy','Fruits','Vegetables',
                'Grocery','Snacks','Dairy','Fruits','Vegetables'],
   'Price': [50,30,45,60,40,80,25,55,70,35,90,20,65,75,45,100,35,50,80,30],
   'Quantity_Sold': [20,35,18,25,30,15,40,16,22,28,12,45,14,20,26,10,32,19,18,35],
   'Discount': [5,3,2,5,4,6,3,2,5,4,6,2,3,5,4,7,3,2,6,3],
   'Revenue': [1000,1050,810,1500,1200,1200,1000,880,1540,980,
               1080,900,910,1500,1170,1000,1120,950,1440,1050]
}

df = pd.DataFrame(data)
# 1. Identify number of rows and columns
print("\n1. NUMBER OF ROWS AND COLUMNS")
print(df.shape)
# 2. Check data types
print("\n2. DATA TYPES")
print(df.dtypes)
# 3. Find missing values and duplicates
print("\n3. MISSING VALUES")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
# Data cleaning
df = df.drop_duplicates()
df = df.dropna()
# 4. Statistical summary
print("\n4. STATISTICAL SUMMARY")
print(df[['Price', 'Quantity_Sold', 'Discount', 'Revenue']].agg(
   ['mean', 'median', 'min', 'max', 'std']
).round(2))

# 5(a). Bar chart
df.groupby('Category')['Revenue'].sum().plot(
   kind='bar', color='skyblue'
)
plt.title("Total Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Total Revenue")
plt.tight_layout()
plt.show()
# 5(b). Histogram
plt.hist(df['Revenue'], bins=6, edgecolor='black')
plt.title("Distribution of Revenue")
plt.xlabel("Revenue")
plt.ylabel("Number of Products")
plt.show()
# 5(c). Scatter plot
```

```
sns.scatterplot(data=df, x='Quantity_Sold', y='Revenue',
                hue='Category')
plt.title("Quantity Sold vs Revenue")
plt.tight_layout()
plt.show()
# 5(d). Box plot
sns.boxplot(data=df, x='Category', y='Revenue')
plt.title("Revenue by Category")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# 6. Identify important patterns
print("\n6. IMPORTANT PATTERNS")
print("Total revenue by category:")
print(df.groupby('Category')['Revenue'].sum().sort_values(ascending=False))

print("\nAverage quantity sold by category:")
print(df.groupby('Category')['Quantity_Sold'].mean().round(2))

print("\nCorrelation:")
print(df[['Price', 'Quantity_Sold', 'Discount', 'Revenue']].corr().round(2))

print("\nProduct with highest quantity sold:")
print(df.loc[df['Quantity_Sold'].idxmax()])

print("\nProduct with highest revenue:")
print(df.loc[df['Revenue'].idxmax()])
# 7. Observations
print("\n7. OBSERVATIONS")
print("1. The dataset contains 20 product records.")
print("2. Fruits generate the highest total category revenue.")
print("3. Product PR12 has the highest quantity sold, with 45 units.")
print("4. Product PR09 has the highest revenue, at 1540.")
print("5. Revenue varies across product categories.")
# 8. Conclusion
print("\n8. CONCLUSION")
print("EDA helps identify high-selling products and revenue patterns.")
print("Fruits generate the highest total revenue in this dataset.")
print("The supermarket can use these findings for inventory planning.")
```

OUTPUTS:

Q1.

```
1. NUMBER OF ROWS AND COLUMNS
(20, 6)
```

Q2.

```
2. DATA TYPES
Product_ID        object
Category          object
Price              int64
Quantity_Sold      int64
Discount           int64
Revenue            int64
dtype: object
```

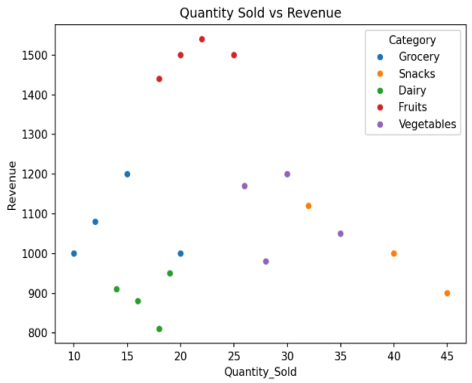
Q3.

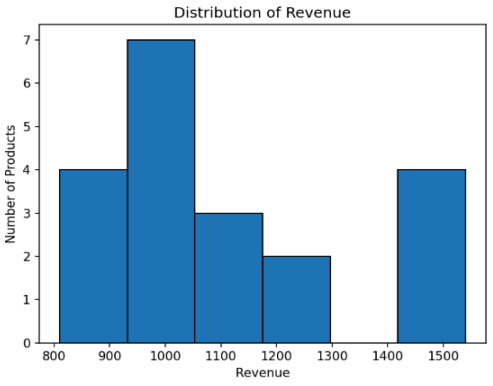
```
3. MISSING VALUES
Product_ID        0
Category          0
Price             0
Quantity_Sold     0
Discount          0
Revenue           0
dtype: int64
Duplicate rows: 0
```

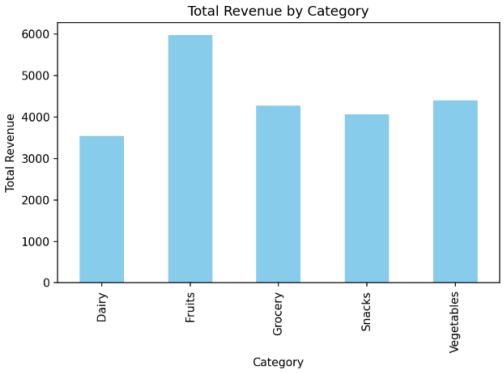
Q4.

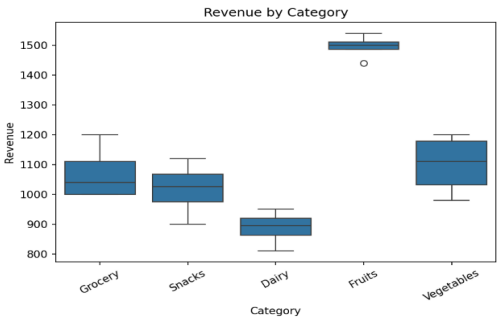
```
4. STATISTICAL SUMMARY
         Price  Quantity_Sold  Discount  Revenue
mean      54.0          24.00      4.00  1114.00
median    50.0          21.00      4.00  1050.00
min       20.0          10.00      2.00   810.00
max      100.0          45.00      7.00  1540.00
std       22.8           9.67      1.56   221.42
```

Q5.









Q6.

```
6. IMPORTANT PATTERNS
Total revenue by category:
Category
Fruits        5980
Vegetables    4400
Grocery       4280
Snacks        4070
Dairy         3550
Name: Revenue, dtype: int64

Average quantity sold by category:
Category
Dairy         16.75
Fruits        21.25
Grocery       14.25
Snacks        38.00
Vegetables    29.75
Name: Quantity_Sold, dtype: float64

Correlation:
               Price  Quantity_Sold  Discount  Revenue
Price           1.00          -0.87      0.79     0.39
Quantity_Sold  -0.87           1.00     -0.51    -0.08
Discount        0.79          -0.51      1.00     0.58
Revenue         0.39          -0.08      0.58     1.00
```

```
Product with highest quantity sold:
Product_ID          PR12
Category          Snacks
Price                 20
Quantity_Sold         45
Discount               2
Revenue              900
Name: 11, dtype: object

Product with highest revenue:
Product_ID          PR09
Category          Fruits
Price                 70
Quantity_Sold         22
Discount               5
Revenue             1540
Name: 8, dtype: object
```

Q7.

```
7. OBSERVATIONS
1. The dataset contains 20 product records.
2. Fruits generate the highest total category revenue.
3. Product PR12 has the highest quantity sold, with 45 units.
4. Product PR09 has the highest revenue, at 1540.
5. Revenue varies across product categories.
```

Q8.

```
8. CONCLUSION
EDA helps identify high-selling products and revenue patterns.
Fruits generate the highest total revenue in this dataset.
The supermarket can use these findings for inventory planning.
```

QUESTION 4:Banking Loan Approval Dataset

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```
# Banking Loan Approval Dataset
data = {
   'Customer_ID': ['C01','C02','C03','C04','C05','C06','C07','C08','C09','C10',
                   'C11','C12','C13','C14','C15','C16','C17','C18','C19','C20'],
   'Income': [45000,25000,60000,22000,50000,30000,70000,28000,55000,24000,
              65000,35000,48000,26000,75000,32000,52000,27000,68000,23000],
   'Credit_Score': [720,580,750,560,710,600,780,590,730,570,
                    760,620,700,585,790,610,725,575,770,555],
   'Loan_Amount':
[200000,150000,300000,120000,250000,180000,350000,160000,270000,130000,
                  320000,200000,240000,140000,400000,190000,260000,150000,330000,125000],
   'Employment_Type': ['Salaried','Self-employed','Salaried','Unemployed','Salaried',
                       'Self-employed','Salaried','Self-employed','Salaried','Unemployed',
                       'Salaried','Self-employed','Salaried','Unemployed','Salaried',
                       'Self-employed','Salaried','Self-employed','Salaried','Unemployed'],
   'Loan_Status': ['Approved','Rejected','Approved','Rejected','Approved',
                   'Rejected','Approved','Rejected','Approved','Rejected',
                   'Approved','Rejected','Approved','Rejected','Approved',
                   'Rejected','Approved','Rejected','Approved','Rejected']
}

df = pd.DataFrame(data)
# 1. Identify number of rows and columns
print("\n1. NUMBER OF ROWS AND COLUMNS")
print("Rows and columns:", df.shape)
# 2. Check data types
print("\n2. DATA TYPES")
print(df.dtypes)
# 3. Find missing values and duplicates
print("\n3. MISSING VALUES")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
# Data cleaning
df = df.drop_duplicates()
df = df.dropna()
# 4. Calculate statistical summary
print("\n4. STATISTICAL SUMMARY")
print(df[['Income', 'Credit_Score', 'Loan_Amount']].agg(
   ['mean', 'median', 'min', 'max', 'std']
).round(2))
# 5(a). Bar chart
df.groupby('Loan_Status')['Income'].mean().plot(kind='bar')
plt.title("Average Income by Loan Status")
plt.xlabel("Loan Status")
plt.ylabel("Average Income")
plt.tight_layout()
plt.show()
# 5(b). Histogram
plt.hist(df['Credit_Score'], bins=6, edgecolor='black')
```

```
plt.title("Distribution of Credit Scores")
plt.xlabel("Credit Score")
plt.ylabel("Number of Customers")
plt.show()
# 5(c). Scatter plot
sns.scatterplot(data=df, x='Income', y='Credit_Score',
                hue='Loan_Status')
plt.title("Income vs Credit Score")
plt.tight_layout()
plt.show()
# 5(d). Box plot
sns.boxplot(data=df, x='Loan_Status', y='Loan_Amount')
plt.title("Loan Amount by Loan Status")
plt.show()
# 6. Identify important patterns
print("\n6. IMPORTANT PATTERNS")
print("\nAverage values by loan status:")
print(df.groupby('Loan_Status')[
    ['Income', 'Credit_Score', 'Loan_Amount']
].mean().round(2))
print("\nLoan status by employment type:")
print(pd.crosstab(df['Employment_Type'], df['Loan_Status']))
print("\nCorrelation:")
print(df[['Income', 'Credit_Score', 'Loan_Amount']].corr().round(2))
# 7. Observations
print("\n7. OBSERVATIONS")
print("1. The dataset contains 20 customer records.")
print("2. Ten loans are approved and ten are rejected.")
print("3. Approved customers have higher incomes in this dataset.")
print("4. Approved customers have higher credit scores in this dataset.")
print("5. All salaried customers are approved in this sample.")
# 8. Conclusion
print("\n8. CONCLUSION")
print("EDA identifies patterns between income, credit score,")
print("employment type, loan amount, and loan approval status.")
print("These patterns describe this dataset and do not establish")
print("universal rules for loan approval.")
```

OUTPUTS:

Q1.

```
1. NUMBER OF ROWS AND COLUMNS
Rows and columns: (20, 6)
```

Q2.

```
2. DATA TYPES
Customer_ID        object
Income              int64
Credit_Score        int64
Loan_Amount         int64
Employment_Type    object
Loan_Status        object
dtype: object
```

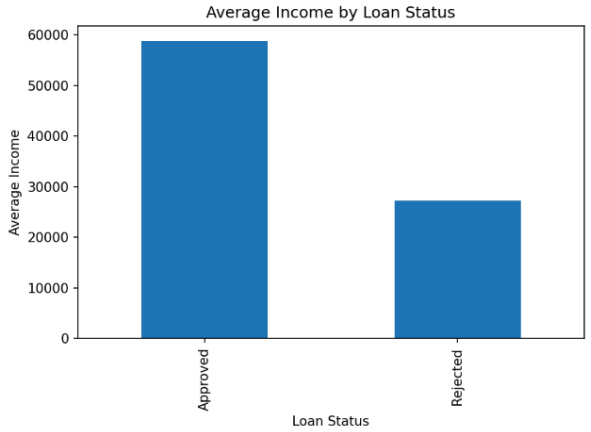
Q3.

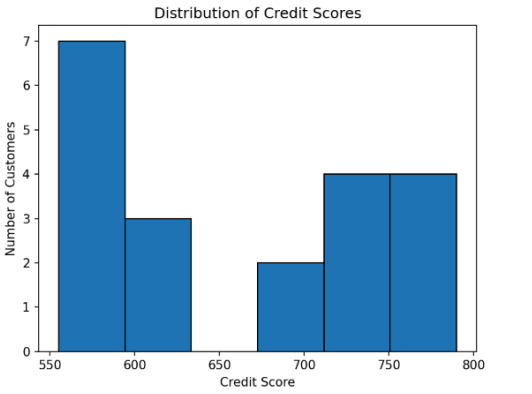
```
3. MISSING VALUES
Customer_ID          0
Income               0
Credit_Score         0
Loan_Amount          0
Employment_Type      0
Loan_Status          0
dtype: int64
Duplicate rows: 0
```

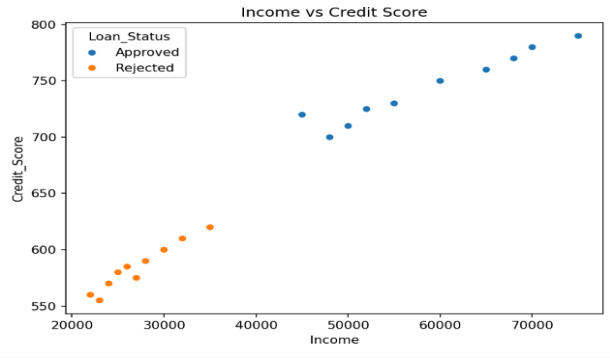
Q4.

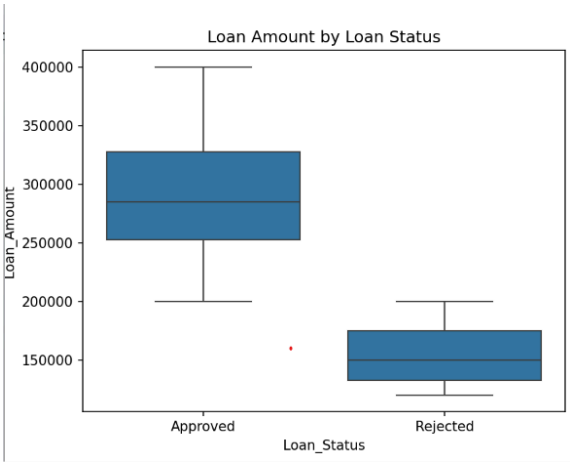
```
4. STATISTICAL SUMMARY
              Income  Credit_Score  Loan_Amount
mean     43000.00        664.00     223250.00
median   40000.00        660.00     200000.00
min      22000.00        555.00     120000.00
max      75000.00        790.00     400000.00
std      17923.82         85.53      83733.21
```

Q5.









Q6.

```
6. IMPORTANT PATTERNS

Average values by loan status:
              Income  Credit_Score  Loan_Amount
Loan_Status
Approved     58800.0         743.5     292000.0
Rejected     27200.0         584.5     154500.0

Loan status by employment type:
Loan_Status      Approved  Rejected
Employment_Type
Salaried               10         0
Self-employed           0         6
Unemployed              0         4

Correlation:
              Income  Credit_Score  Loan_Amount
Income          1.00          0.98         0.99
Credit_Score    0.98          1.00         0.95
Loan_Amount     0.99          0.95         1.00
```

Q7.

```
7. OBSERVATIONS
1. The dataset contains 20 customer records.
2. Ten loans are approved and ten are rejected.
3. Approved customers have higher incomes in this dataset.
4. Approved customers have higher credit scores in this dataset.
5. All salaried customers are approved in this sample.
```

Q8.

```
8. CONCLUSION
EDA identifies patterns between income, credit score,
employment type, loan amount, and loan approval status.
These patterns describe this dataset and do not establish
universal rules for loan approval.
```

QUESTION 5:Agriculture Crop Yield Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Agriculture Crop Yield Dataset
data = {
    'Farm_ID': ['F01','F02','F03','F04','F05','F06','F07','F08','F09','F10',
                'F11','F12','F13','F14','F15','F16','F17','F18','F19','F20'],
    'Rainfall_mm':
[850,600,900,500,780,650,920,720,550,800,880,620,760,940,580,820,700,960,540,790],
    'Temperature': [28,32,27,34,29,31,26,30,33,28,27,32,29,26,33,28,30,25,34,29],
    'Fertilizer_kg': [60,45,65,40,55,48,70,52,42,58,62,46,54,72,44,60,50,75,41,56],
    'Soil_Type': ['Loamy','Sandy','Loamy','Sandy','Clay','Sandy','Loamy','Clay','Sandy','Loamy',
                  'Loamy','Sandy','Clay','Loamy','Sandy','Clay','Clay','Loamy','Sandy','Clay'],
    'Crop_Yield_kg': [3200,2100,3500,1800,3000,2300,3700,2800,1900,3100,
                      3400,2200,2950,3800,2000,3150,2700,3900,1850,3050]
}

df = pd.DataFrame(data)
```

```
# 1. Number of rows and columns
print("\n1. NUMBER OF ROWS AND COLUMNS")
print("Rows and columns:", df.shape)
# 2. Check data types
print("\n2. DATA TYPES")
print(df.dtypes)
# 3. Find missing values and duplicates
print("\n3. MISSING VALUES")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
# Data cleaning
df = df.drop_duplicates()
df = df.dropna()
# 4. Statistical summary
print("\n4. STATISTICAL SUMMARY")
print(df[['Rainfall_mm', 'Temperature', 'Fertilizer_kg',
        'Crop_Yield_kg']].agg(
   ['mean', 'median', 'min', 'max', 'std']
).round(2))
# 5(a). Bar chart
df.groupby('Soil_Type')['Crop_Yield_kg'].mean().plot(kind='bar')
plt.title("Average Crop Yield by Soil Type")
plt.xlabel("Soil Type")
plt.ylabel("Average Crop Yield (kg)")
plt.tight_layout()
plt.show()
# 5(b). Histogram
plt.hist(df['Crop_Yield_kg'], bins=6, edgecolor='black')
plt.title("Distribution of Crop Yield")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Number of Farms")
plt.show()
# 5(c). Scatter plot
sns.scatterplot(data=df, x='Rainfall_mm', y='Crop_Yield_kg',
              hue='Soil_Type')
plt.title("Rainfall vs Crop Yield")
plt.tight_layout()
plt.show()
# 5(d). Box plot
sns.boxplot(data=df, x='Soil_Type', y='Crop_Yield_kg')
plt.title("Crop Yield by Soil Type")
plt.show()
# 6. Identify important patterns
print("\n6. IMPORTANT PATTERNS")
print("\nAverage crop yield by soil type:")
print(df.groupby('Soil_Type')['Crop_Yield_kg'].mean().round(2))
print("\nCorrelation:")
print(df[['Rainfall_mm', 'Temperature', 'Fertilizer_kg',
        'Crop_Yield_kg']].corr().round(2))
```

```
print("\nFarm with highest crop yield:")
print(df.loc[df['Crop_Yield_kg'].idxmax()])
# 7. Observations
print("\n7. OBSERVATIONS")
print("1. The dataset contains 20 farm records.")
print("2. Farm F18 has the highest crop yield of 3900 kg.")
print("3. Loamy soil generally produces higher yields in this dataset.")
print("4. Higher rainfall is generally associated with higher crop yield.")
print("5. Higher fertilizer usage is generally associated with higher crop yield.")
# 8. Conclusion
print("\n8. CONCLUSION")
print("EDA identifies patterns between rainfall, temperature,")
print("fertilizer usage, soil type, and crop yield.")
print("These findings can help guide further crop production analysis.")
```

OUTPUTS:

Q1.

```
1. NUMBER OF ROWS AND COLUMNS
Rows and columns: (20, 6)
```

Q2.

```
2. DATA TYPES
Farm_ID          object
Rainfall_mm       int64
Temperature       int64
Fertilizer_kg     int64
Soil_Type        object
Crop_Yield_kg     int64
dtype: object
```

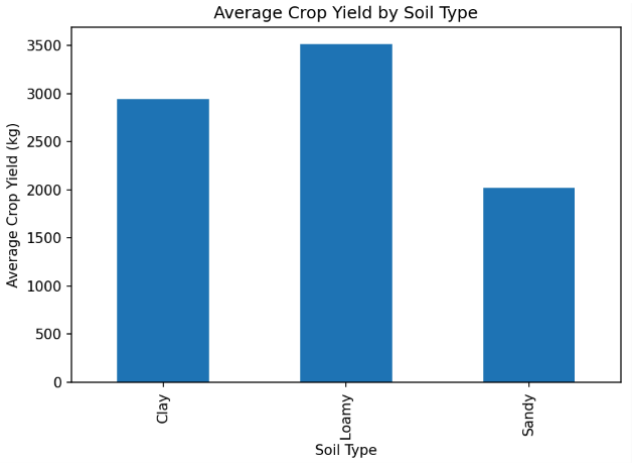
Q3.

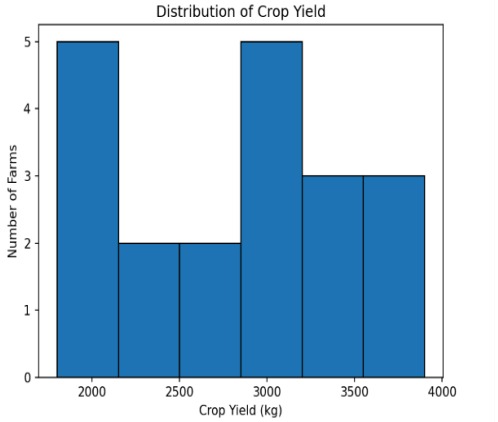
```
3. MISSING VALUES
Farm_ID          0
Rainfall_mm      0
Temperature      0
Fertilizer_kg    0
Soil_Type        0
Crop_Yield_kg    0
dtype: int64
Duplicate rows: 0
```

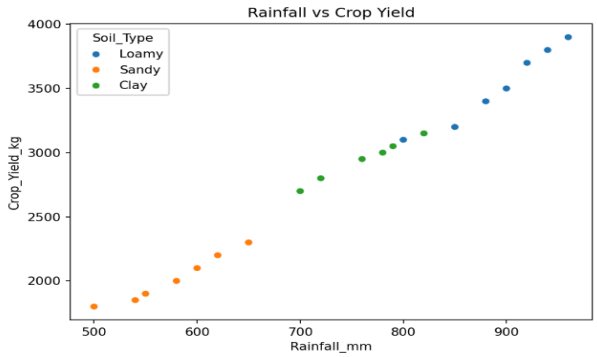
Q4.

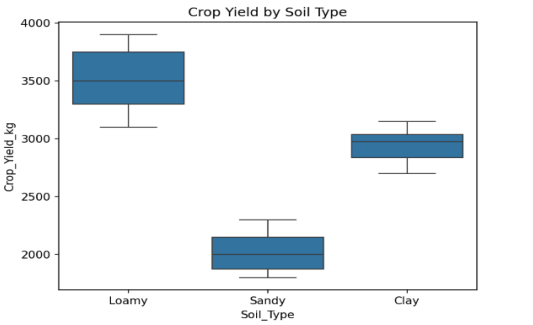
```
4. STATISTICAL SUMMARY
        Rainfall_mm  Temperature  Fertilizer_kg  Crop_Yield_kg
mean         743.00        29.55          54.75        2820.00
median       770.00        29.00          54.50        2975.00
min          500.00        25.00          40.00        1800.00
max          960.00        34.00          75.00        3900.00
std          144.34         2.76          10.51         681.41
```

Q5.









Q6.

```
6. IMPORTANT PATTERNS

Average crop yield by soil type:
Soil_Type
Clay     2941.67
Loamy    3514.29
Sandy    2021.43
Name: Crop_Yield_kg, dtype: float64

Correlation:
              Rainfall_mm  Temperature  Fertilizer_kg  Crop_Yield_kg
Rainfall_mm          1.00        -0.99           0.98           1.00
Temperature         -0.99         1.00          -0.98          -0.99
Fertilizer_kg        0.98        -0.98           1.00           0.98
Crop_Yield_kg        1.00        -0.99           0.98           1.00

Farm with highest crop yield:
Farm_ID            F18
Rainfall_mm        960
Temperature         25
Fertilizer_kg       75
Soil_Type        Loamy
Crop_Yield_kg     3900
Name: 17, dtype: object
```

Q7.

```
7. OBSERVATIONS
1. The dataset contains 20 farm records.
2. Farm F18 has the highest crop yield of 3900 kg.
3. Loamy soil generally produces higher yields in this dataset.
4. Higher rainfall is generally associated with higher crop yield.
5. Higher fertilizer usage is generally associated with higher crop yield.
```

Q8.

```
8. CONCLUSION
EDA identifies patterns between rainfall, temperature,
fertilizer usage, soil type, and crop yield.
These findings can help guide further crop production analysis.
```